Universidad de Chile
Facultad de Ciencias Físicas y Matemáticas
Departamento de Ciencias de la Computación

Project Manual: Game of Life on CPU and GPU

Computación en GPU

Author: Matías Saavedra

Document: developer manual
Last updated: Friday 5th June, 2026
Santiago de Chile

Resumen

This document is the developer manual for the project: three implementations of Conway's Game of Life — a sequential CPU baseline, a CUDA backend and an OpenCL backend — built to benchmark cells-evaluated-per-second and report the GPU speed-up over the CPU. It explains the architecture from first principles, states the single invariant that shapes every design decision (*the three backends must compute byte-for-byte identical boards so that any difference in speed is the real result*), and then walks the source code function by function, with the C++ files explained line by line. At the time of writing the CPU backend is complete and acts as the correctness oracle; the CUDA and OpenCL backends are scaffolded but not yet implemented. The headline metric is fixed throughout: Mcells/s = (rows $\times$ cols $\times$ gens)/($t \times 10^6$), measured against a no-op renderer so that rendering and host $\leftrightarrow$ device transfers never pollute the number.

Índice de Contenidos

Índice de Tablas

Índice de Códigos

1. What this manual is, and how to read it

This is the engineering manual for the codebase, not the graded report. The graded report lives in `main.tex` and presents experiments and results; this manual explains *how the code works and why it is built the way it is*. Read it top to bottom the first time: the early sections establish the one constraint that everything else follows from, and the later sections assume you already understand it.

The manual blends two styles. It is structured like an engineering report (setup, mechanism, tables, explicit trade-offs) but written like a deep code explanation (every non-obvious decision is traced to *why it exists* and *what would break if it were different*). For the C++ translation units it descends to a line-by-line account; for the headers, which are mostly declarations, it summarises and shows the parts that carry real logic.

Build.

The manual compiles with the same vendored template as the report:

Código 1: Building this manual.

```
cd report && latexmk -pdf manual.tex
# or: pdflatex manual.tex && pdflatex manual.tex   (second pass for the TOC)
```

2. The problem, and the invariant it forces

The assignment looks like a simulation task but is really a *measurement* task. The deliverable is a number and an explanation of that number: how many cells per second each backend evaluates, and *why* the GPU wins (or loses) at a given grid size. Conway's Game of Life — here the HighLife variant `B36/S23` — is only the workload. It is deliberately a good workload for the question being asked: a nine-point stencil over one-byte cells is arithmetically trivial and memory-bound, so the experiment isolates memory bandwidth and launch/transfer overhead, which is exactly the interesting story for a GPU report.

Every architectural decision in the codebase serves one goal:

> *Run the identical computation on three backends and trust that any difference in output is a bug, so that any difference in speed is the real result.*

Two hard requirements fall out of that sentence, and they explain the entire design:

1. **Bit-for-bit equivalence across backends.** If CPU and GPU produce different boards, the cells/sec comparison is meaningless. This forces a *shared* cell layout, a *shared* rule, and *shared* edge handling.

2. **Measurement that is not polluted by I/O.** The headline metric is compute throughput, so rendering and host $\leftrightarrow$ device transfers must be removable from the timed path.

The rule variant is the one detail most likely to be implemented inconsistently, so it is worth stating precisely. It is *not* vanilla Conway:

- a dead cell is born on **exactly 3 OR exactly 6** live neighbours;
- a live cell survives on 2 or 3 live neighbours;
- everything else dies or stays dead.

The "birth on 6" clause is the easy mistake. It must hold identically in CPU, CUDA and OpenCL, which is why it is written exactly once (Section 4.2) and pinned by a dedicated test (Section 11).

3. Architecture at a glance

The codebase is a **strategy pattern**: a backend-agnostic core plus two swappable interfaces. The only thing that genuinely differs between backends — the per-generation compute — sits behind `ISimEngine`; output sits behind `IRenderer`; the application owns the loop and wires one of each at runtime from the parsed configuration.

Código 2: The strategy layout. Parenthesised backends are scaffolded, not yet implemented.

```
Application (main.cpp)  -- owns the loop + Config/CLI
  |-- ISimEngine   <- CpuEngine | (CudaEngine) | (OpenCLEngine)
  |-- IRenderer    <- NullRenderer | TextRenderer | (GuiRenderer later)
  '-- Grid + Pattern  (shared, backend-agnostic state & seed data)
```

The dependency arrows point *from* engines *to* the core, never back — that one-directional dependency is what keeps the core backend-agnostic and lets a single host-side harness drive all three engines. The link graph is:

Código 3: Library dependency graph (CMake targets).

```
gol_core  (Grid, Config+CLI, patterns, RLE, interfaces)
   |-- gol_cpu     (CpuEngine; links Threads::Threads)
   |-- gol_render  (TextRenderer)
   '-- (gol_cuda, gol_opencl -- opt-in, not yet present)
gol  executable  =  gol_core + gol_cpu + gol_render
```

Table 1 is the file map. Header-only units carry no `.cpp` and impose no link dependency.

Tabla 1: File map and status. "HO" = header-only (logic lives entirely in the header). The "$\mathcal{S}$" column points to the section that explains it.

File	Responsibility	$\mathcal{S}$
include/gol/Grid.hpp	Board state; the shared layout (HO)	4.1
include/gol/LifeRules.hpp	The variant rule, written once (HO)	4.2
include/gol/Timer.hpp	Monotonic host stopwatch (HO)	4.3
include/gol/ISimEngine.hpp	Engine strategy interface (HO)	4.4
include/gol/IRenderer.hpp	Renderer strategy interface (HO)	4.4
include/gol/Config.hpp	Run configuration struct (HO)	4.5
include/gol/engines/CpuEngine.hpp	CPU engine declaration	7
include/gol/render/NullRenderer.hpp	No-op renderer (HO)	9
include/gol/render/TextRenderer.hpp	Text renderer declaration	9
include/gol/render/AnsiRenderer.hpp	In-place ANSI renderer decl.	9
include/gol/patterns/Pattern.hpp	Sparse seed pattern (HO)	8
include/gol/patterns/RleLoader.hpp	RLE loader declaration	8
src/core/Config.cpp	CLI parsing	5
src/core/main.cpp	The loop and the metric	6
src/engines/cpu/CpuEngine.cpp	CPU engine (seq + parallel)	7
src/patterns/RleLoader.cpp	RLE decoding	8
src/patterns/Pattern.cpp	Stamping a pattern onto a grid	8
src/render/TextRenderer.cpp	ASCII frame dump	9
src/render/AnsiRenderer.cpp	In-place ANSI animation	9
CMakeLists.txt	Build, GPU opt-in gating	10
tests/*.cpp	Executable spec of the invariants	11

4. The shared core

The core is everything that must be identical across backends, plus the glue that makes the backends interchangeable.

4.1. Grid — the spine of cross-backend equivalence

`Grid` is the single most important object in the project. It is a **flat, row-major `std::vector<unsigned char>`** where cell (x, y) lives at index $y \cdot \text{cols} + x$, with 0 = dead, 1 = alive.

Código 4: include/gol/Grid.hpp — the load-bearing parts.

```
class Grid {
public:
  Grid(std::size_t rows, std::size_t cols, unsigned char fillValue = 0)
```

```
    : rows_(rows), cols_(cols), cells_(rows * cols, fillValue) {}

  // Unchecked access by (x, y): the hot path must stay branch-free.
  unsigned char& at(std::size_t x, std::size_t y) { return cells_[y * cols_ + x]; }
  unsigned char  at(std::size_t x, std::size_t y) const { return cells_[y * cols_ + x]; }

  unsigned char* data() { return cells_.data(); }          // for host<->device transfers

  // Deterministic seeding: same seed => same board across all backends.
  void randomize(std::uint64_t seed, double aliveProbability = 0.3) {
    std::mt19937_64 rng(seed);
    std::bernoulli_distribution alive(aliveProbability);
    for (auto& c : cells_) c = alive(rng) ? 1u : 0u;
  }

  // Bit-for-bit comparison: the equivalence-test oracle check.
  bool operator==(const Grid& other) const {
    return rows_ == other.rows_ && cols_ == other.cols_ && cells_ == other.cells_;
  }
private:
  std::size_t rows_, cols_;
  std::vector<unsigned char> cells_; // row-major, size rows_ * cols_
};
```

Why this exact representation.

Each choice is forced by the equivalence requirement:

- **Flat and contiguous**, because a GPU device buffer is a flat array. A flat `unsigned char` buffer maps to `cudaMemcpy` / `clEnqueueWriteBuffer` with *zero* translation — the same bytes on host and device. A `vector<vector<»` would need a transform on every transfer, and the transform itself could introduce divergence.
- **One byte per cell, not one bit.** Bit-packing would be 8× smaller and use less bandwidth, but it would turn the neighbour read into a bit-extraction and make the three backends harder to keep identical. The project accepts the 8× memory cost to keep the stencil a plain array read. This is also *why* shared memory is expected to help little on the GPU (Section 12): a one-byte stencil already fits comfortably in cache.
- **`at(x,y)` is unchecked.** The simulation hot path evaluates this billions of times; a bounds branch per access would be a measurable tax on a memory-bound kernel. Bounds are the caller's responsibility.

The invariant.

`Grid` owns no device memory and contains no device code. It is pure host state. Engines own their own ping-pong buffers and touch the `Grid` only through `data()` during

`upload()`/`download()`. **What would break otherwise:** if the `Grid` held device pointers, the host harness and the equivalence test could no longer drive all three engines through one type.

Two methods exist purely for equivalence. `randomize` uses a seeded `std::mt19937_64`, so "same seed ⇒ same starting board" holds across backends — without a deterministic seed you could not start CPU and GPU from an identical state. `operator==` compares dimensions *and* the full byte vector; this is literally the oracle check the cross-backend test will call.

4.2. LifeRules — the rule written once, compiled three times

The rule variant is written exactly once and shared by source between the CPU and CUDA backends.

Código 5: include/gol/LifeRules.hpp — the single source of truth for the rule.

```
#ifdef ___CUDACC___
#define GOL_FN ___host___ ___device___ inline
#else
#define GOL_FN inline
#endif

namespace gol {
// VARIANT (NOT vanilla Conway):
//   dead cell born on EXACTLY 3 OR EXACTLY 6 neighbours;
//   live cell survives on 2 or 3; everything else dies.
GOL_FN unsigned char nextState(unsigned char alive, int neighbors) {
  if (alive) {
    return (neighbors == 2 || neighbors == 3) ? 1u : 0u; // survive
  }
  return (neighbors == 3 || neighbors == 6) ? 1u : 0u;   // birth (3 or 6)
}
} // namespace gol
```

The mechanism.

When the host C++ compiler reads this file, `nextState` is an ordinary `inline` function used by `CpuEngine`. When `nvcc` reads the *same* file it defines `___CUDACC___`, so the macro expands to `___host___ ___device___` and the CUDA kernel calls the byte-for-byte identical logic. CPU and CUDA therefore *cannot* diverge on the rule — they share source.

The acknowledged exception.

OpenCL C cannot `#include` a C++ header, so the plan is to mirror this function in `kernel.cl` (read at runtime, kept diff-able against this header). The project deliberately does

not chase zero duplication between CUDA and OpenCL: two duplicated copies that are easy to eyeball-diff beat one clever abstraction that is hard to follow. That is a stated trade-off, not an oversight.

4.3. Timer — monotonic host timing

Código 6: include/gol/Timer.hpp — the host stopwatch.

```
class Timer {
public:
  Timer() : start_(Clock::now()) {}
  void reset() { start_ = Clock::now(); }
  double elapsedMillis() const {
    return std::chrono::duration<double, std::milli>(Clock::now() - start_).count();
  }
private:
  using Clock = std::chrono::steady_clock;   // monotonic, not system_clock
  Clock::time_point start_;
};
```

It uses `steady_clock`, not `system_clock`, because it is monotonic: it cannot jump backwards if the wall clock is adjusted mid-run, which would corrupt a timing. This is the CPU backend's kernel-timing source; the GPU backends will time with `cudaEvent_t` / CL events instead.

4.4. ISimEngine and IRenderer — the two strategy interfaces

Código 7: include/gol/ISimEngine.hpp — the engine contract.

```
class ISimEngine {
public:
  virtual ~ISimEngine() = default;
  virtual void upload(const Grid& initial) = 0;   // host -> "current" device buffer
  virtual void step() = 0;                        // read current, write next, swap
  virtual void download(Grid& out) = 0;           // device -> host
  virtual double lastKernelMillis() const = 0;    // compute-only time of last step()
  virtual std::string name() const = 0;           // "cpu", "cuda", ...
};
```

The crucial part is the documented **ping-pong contract**: an engine keeps two equally-sized buffers; `step()` reads the "current" buffer, writes the "next" buffer, then swaps them. This guarantees `step()` advances exactly one generation and never aliases its input and output.

Why the two-buffer rule is non-negotiable.

A stencil reads each cell's neighbours. If `step()` wrote new values into the same buffer it is still reading from, cells processed later in the sweep would see *next-generation* neighbours instead of *current-generation* ones, and the result would silently depend on traversal order. The two-buffer invariant prevents that, and it is stated in the interface so every backend obeys it. GPU engines do the swap on the device; only `upload`/`download` cross the bus.

Código 8: include/gol/IRenderer.hpp — the output contract.

```
class IRenderer {
public:
  virtual ~IRenderer() = default;
  virtual void render(const Grid& grid, std::uint64_t generation) = 0;
  virtual bool shouldClose() const { return false; } // interactive renderers override
};
```

`shouldClose()` defaults to `false`; only an interactive GUI renderer would override it to break the loop early. The key implementation is `NullRenderer` (Section 9), whose entire purpose is to *not exist* in the timed path.

4.5. Config — the run configuration

Código 9: include/gol/Config.hpp — the knobs (abridged).

```
enum class EngineKind { Cpu, Cuda, OpenCL };
enum class RendererKind { Null, Text, Ansi };

struct Config {
  std::size_t rows = 256, cols = 256;
  std::uint64_t generations = 100;
  EngineKind engine = EngineKind::Cpu;
  RendererKind renderer = RendererKind::Null;
  unsigned threads = 1;       // 1=sequential, N=parallel, 0=all hardware cores
  int blockSize = 256;        // GPU: threads per block (swept in the report)
  bool useShared = false;     // GPU: shared/local-memory tiling
  bool wrap = false;          // false=bounded edges, true=toroidal
  std::uint64_t seed = 1;
  std::optional<std::string> rlePath;
};
Config parse(int argc, char** argv);
```

The GPU-only knobs (`blockSize`, `useShared`) live here and are simply ignored by `CpuEngine`. That is deliberate: one `Config` type, and a single binary whose flag surface is the *union* over all backends, can script the entire benchmark matrix (block sizes $\{32, 64, 128, 256\}$ × shared/no-shared) without recompilation.

5. Configuration parsing — Config.cpp

This translation unit turns `argv` into a fully-resolved `Config`. Three file-private helpers carry the logic.

Código 10: src/core/Config.cpp — value consumption and integer parsing.

```
[[noreturn]] void usageAndExit(int code) { /* prints help */ std::exit(code); }

std::string nextValue(int argc, char** argv, int& i, const char* flag) {
  if (i + 1 >= argc) { std::cerr << "error: " << flag << " requires a value\n";
                       usageAndExit(2); }
  return argv[++i];                    // consume the value AND advance caller's i
}

unsigned long long toULL(const std::string& s, const char* flag) {
  try { return std::stoull(s); }
  catch (const std::exception&) {
    std::cerr << "error: " << flag << " expects an integer, got '" << s << "'\n";
    usageAndExit(2);
  }
}
```

`usageAndExit` is marked `[[noreturn]]`: it never returns (it ends in `std::exit`), which lets call sites fall through after calling it without the compiler complaining about a missing return value.

`nextValue` is the mechanism that makes the parse loop correct. It takes the loop index `i` **by reference**; line 36 checks a token follows the flag, and line 40 *pre-increments* `i` — this both returns the value token and advances the caller's index past it, so the main `for` loop does not re-examine a flag's argument as if it were another flag. Remove the `&` on `i` and the parser would try to interpret every value as a flag.

`toULL` centralises integer parsing: `std::stoull` throws `std::invalid_argument` on non-numeric input and `std::out_of_range` on overflow; both are caught to print a friendly message and exit, instead of an uncaught exception aborting the program.

Código 11: src/core/Config.cpp — the dispatch loop (representative branches).

```
Config parse(int argc, char** argv) {
  Config cfg;                              // start from defaults
  for (int i = 1; i < argc; ++i) {         // skip argv[0]
    const std::string arg = argv[i];
    if (arg == "-h" || arg == "--help") usageAndExit(0);
    else if (arg == "-r" || arg == "--rows")
      cfg.rows = (std::size_t)toULL(nextValue(argc, argv, i, "--rows"), "--rows");
    else if (arg == "--wrap")  cfg.wrap = true;          // boolean flag, no value
    else if (arg == "--rle")   cfg.rlePath = nextValue(argc, argv, i, "--rle");
```

```
    else if (arg == "--engine") {
      const std::string v = nextValue(argc, argv, i, "--engine");
      if (v == "cpu") cfg.engine = EngineKind::Cpu;
      else if (v == "cuda") cfg.engine = EngineKind::Cuda;
      else if (v == "opencl") cfg.engine = EngineKind::OpenCL;
      else { std::cerr << "error: unknown engine '" << v << "'\n"; usageAndExit(2); }
    }
    // ... --cols --gens --threads --seed --renderer --block --shared ...
    else { std::cerr << "error: unknown option '" << arg << "'\n"; usageAndExit(2); }
  }
  return cfg;
}
```

Each value-taking branch uses the idiom `toULL(nextValue(...))`, which consumes and parses in one expression. `–wrap`, `–shared` are value-less booleans. `–block`/`–shared` are accepted even in a CPU-only build — that is the union-flag-surface point in code.

What can go wrong.

Missing value, non-numeric value, unknown flag, and unknown engine/renderer name are all caught. One unguarded edge: a negative number passed to `–rows` is accepted by `stoull` as a huge unsigned value (a known `stoull` quirk); it would surface immediately as a failed allocation.

6. The application loop and the measurement invariant — main.cpp

`main.cpp` owns the loop, wires the chosen engine and renderer, seeds the board, runs the generations, and prints the metric. It is where requirement 2 from Section 2 — keeping I/O out of the number — is enforced.

Código 12: src/core/main.cpp — the factory and the seeder.

```
std::unique_ptr<IRenderer> makeRenderer(RendererKind kind) {
  switch (kind) {
    case RendererKind::Text: return std::make_unique<TextRenderer>();
    case RendererKind::Ansi: return std::make_unique<AnsiRenderer>();
    case RendererKind::Null:
    default: return std::make_unique<NullRenderer>();   // safe default = benchmark-safe
  }
}

void seed(Grid& grid, const Config& cfg) {
  if (cfg.rlePath) {
    Pattern p = RleLoader::load(*cfg.rlePath);
    const std::size_t ox = cfg.cols > p.width  ? (cfg.cols - p.width)  / 2 : 0;
```

```
    const std::size_t oy = cfg.rows > p.height ? (cfg.rows - p.height) / 2 : 0;
    p.applyTo(grid, ox, oy);                          // centre the pattern
  } else {
    grid.randomize(cfg.seed);                         // deterministic fill
  }
}
```

The factory's `default` falling to `NullRenderer` is a safety choice: any unexpected enum value degrades to the benchmark-safe no-op renderer rather than crashing. In `seed`, the ternaries on the origins are *not* cosmetic: `cols`, `width` are unsigned, so if the pattern were wider than the grid, `cfg.cols - p.width` would underflow to a gigantic value and `applyTo` would clip the whole pattern off-grid. The guard makes the origin 0 in that case so at least the top-left lands.

Código 13: src/core/main.cpp — the loop and the metric.

```
engine.upload(grid);

// Generation 0 is the seed itself: render it before the first step so frames
// are labelled seed, step 1, step 2, ... The NullRenderer path skips this.
if (rendering) renderer->render(grid, 0);

Timer wall;
double kernelMs = 0.0;
for (std::uint64_t gen = 0; gen < cfg.generations; ++gen) {
  engine.step();
  kernelMs += engine.lastKernelMillis();
  if (rendering) {                    // <-- the whole I/O path is gated here
    engine.download(grid);
    renderer->render(grid, gen + 1);     // state after gen+1 steps
    if (renderer->shouldClose()) break;
  }
}
const double wallMs = wall.elapsedMillis();

const double cells = (double)cfg.rows * (double)cfg.cols * (double)cfg.generations;
auto mcellsPerSec = [cells](double ms) {
  return ms > 0.0 ? cells / (ms / 1000.0) / 1e6 : 0.0;
};
```

The measurement invariant, in code.

The `rendering` bool (set to `cfg.renderer != RendererKind::Null`) gates the entire `download + render` block. Under `NullRenderer` the loop body is *only* `step()` plus the time accumulation: no transfers, no formatting. That is why the rule "always benchmark against `NullRenderer`" holds — it is structurally enforced, not a convention you have to remember.

Two clocks.

`kernelMs` accumulates each step's backend-native compute time (`lastKernelMillis`); `wall` measures the whole loop. Reporting both separately lets the report distinguish "the kernel got faster" from "we are transfer-bound".

Two subtleties.

The seed is generation 0 and is rendered *before* the first `step()` so frame labels are honest (this is the fix in commit `0f87fd9`); the `(double)` casts on the `cells` product avoid 32-bit overflow at large grids (e.g. $4096^2 \times 1000 \approx 6.9 \times 10^{10}$ overflows 32-bit but is exact in `double`), and the `ms > 0.0` guard avoids a divide-by-zero yielding `inf`/NaN.

The whole body runs inside a `try/catch` so that, e.g., a missing RLE file (which throws in the loader) is reported cleanly on `stderr` with exit code 1. A guard before the loop rejects `–engine cuda|opencl` because those backends are not built yet.

7. The CPU engine — CpuEngine.cpp

This is the most subtle file in the project, and the most important: it is the correctness oracle for every future backend, and it implements the sequential baseline and the data-parallel version *in one code path*. The reason one path can serve both is that the Game of Life step has **no intra-generation data dependencies**: every cell reads only the current buffer and writes only the next, so parallelising is *purely* a matter of splitting the rows across threads. The per-cell math is identical; only the row range differs.

7.1. Data model and invariants

Código 14: include/gol/engines/CpuEngine.hpp — the state (abridged).

```
class CpuEngine final : public ISimEngine {
  // ... interface methods ...
  unsigned threads_;  bool wrap_;
  std::size_t rows_ = 0, cols_ = 0;
  std::vector<unsigned char> cur_;   // current generation (read by step)
  std::vector<unsigned char> nxt_;   // scratch for the next generation
  double lastMs_ = 0.0;
  // Persistent worker pool (only populated when threads_ >= 2):
  std::vector<std::thread> workers_;
  std::unique_ptr<std::barrier<>> barrier_;
  const unsigned char* src_ = nullptr;   // buffers published to workers each step
  unsigned char* dst_ = nullptr;
  std::atomic<bool> stop_{false};
};
```

- `cur_` / `nxt_`: the two ping-pong buffers. Invariant: at the start of every `step()`, `cur_` holds the current generation; at the end, after a swap, it holds the new one.

- `workers_`: a *persistent* thread pool, created only when `threads_ >= 2`. Persistent so per-generation thread *creation* never enters the timed region.
- `barrier_`: a `std::barrier` with exactly `threads_` participants (main thread + `threads_-1` workers).
- `src_` / `dst_`: raw pointers "published" to the workers each step so they know which buffers to read/write.
- `stop_`: an atomic flag used only at destruction.

The class is non-copyable and non-movable: it owns `std::threads` and a `std::barrier`, neither of which has sensible copy/move semantics.

7.2. Thread resolution, construction, and shutdown

Código 15: src/engines/cpu/CpuEngine.cpp — lifecycle.

```
unsigned CpuEngine::resolveThreads(unsigned requested) {
  if (requested != 0) return requested;
  unsigned hw = std::thread::hardware_concurrency();
  return hw == 0 ? 1u : hw;          // hardware_concurrency may report 0
}

CpuEngine::CpuEngine(unsigned threads, bool wrap)
    : threads_(resolveThreads(threads)), wrap_(wrap) {
  if (threads_ >= 2) startPool();     // sequential path touches no threads at all
}

CpuEngine::~CpuEngine() {
  if (!workers_.empty()) {
    stop_.store(true, std::memory_order_relaxed);
    barrier_->arrive_and_wait();      // release parked workers ONE last time
    for (auto& w : workers_) w.join();
  }
}
```

`resolveThreads` maps the request to a concrete count. `0` means "all cores", but `hardware_concurrency()` is allowed to return 0 when it cannot detect the count, so it falls back to 1. This guarantees `threads_ >= 1` always, which everything else relies on (e.g. `rangeFor` divides by it).

The destructor is the trickiest correctness point in the file. At rest, every worker is parked at the phase-1 barrier inside `workerLoop`. To shut down: set `stop_` (relaxed ordering suffices — the barrier provides the happens-before edge that publishes the flag), then `arrive_and_wait()` *once*. That single arrival completes phase 1 and releases every worker, which then sees `stop_` and **returns without arriving at the phase-2 barrier**. That

asymmetry is essential: if the workers hit phase 2 on shutdown, the main thread (which is not waiting on phase 2 here) would leave them blocked and `join()` would deadlock.

7.3. The worker pool and the two-phase handshake

Código 16: src/engines/cpu/CpuEngine.cpp — pool startup and worker loop.

```
void CpuEngine::startPool() {
  // participants = main thread + (threads_ - 1) workers = threads_
  barrier_ = std::make_unique<std::barrier<>>((std::ptrdiff_t)threads_);
  workers_.reserve(threads_ - 1);
  for (unsigned id = 1; id < threads_; ++id)       // id 0 is the main thread
    workers_.emplace_back([this, id] { workerLoop(id); });
}

void CpuEngine::workerLoop(unsigned id) {
  while (true) {
    barrier_->arrive_and_wait();                  // phase 1: wait for work
    if (stop_.load(std::memory_order_relaxed)) return;
    auto [yBegin, yEnd] = rangeFor(id);
    computeRows(src_, dst_, yBegin, yEnd);
    barrier_->arrive_and_wait();                  // phase 2: signal completion
  }
}
```

Worker ids run $1 \ldots \text{threads\_} - 1$; **id 0 is the main thread**. The main thread doing real work (partition 0) rather than just coordinating is a deliberate efficiency choice: with N cores you get N workers, not $N - 1$ plus an idle coordinator.

The worker is a two-phase handshake. **Phase 1** ("go") blocks until `step()` or the destructor arrives. After release the worker checks `stop_` — the only way the loop ever ends. Otherwise it computes its row slice and signals **phase 2** ("done"), then loops back to park at phase 1. The two distinct barriers per cycle separate "all threads have started this generation" from "all threads have finished it".

Why the lock-free pointer publish is safe.

`step()` writes `src_`/`dst_` *before* arriving at phase 1; the worker reads them *after* leaving phase 1. The barrier establishes a happens-before relationship, so the writes are visible without any lock or atomic on the pointers themselves.

7.4. Row partitioning — rangeFor

Código 17: src/engines/cpu/CpuEngine.cpp — the half-open row range for a partition.

```
std::pair<std::size_t, std::size_t> CpuEngine::rangeFor(unsigned id) const {
  const std::size_t base = rows_ / threads_;
  const std::size_t rem  = rows_ % threads_;
  const std::size_t begin = id * base + std::min<std::size_t>(id, rem);
  const std::size_t count = base + (id < rem ? 1u : 0u);
  return {begin, begin + count};
}
```

The first `rem` partitions each get one extra row. Worked example with `rows_ = 17`, `threads_ = 4` (base = 4, rem = 1):

Tabla 2: `rangeFor` over 17 rows, 4 threads. Rows are contiguous, with no gaps or overlaps, and sum to 17.

id	base	extra	begin	range
0	4	1	0	$[0, 5)$
1	4	0	5	$[5, 9)$
2	4	0	9	$[9, 13)$
3	4	0	13	$[13, 17)$

The `min(id, rem)` term is the clever bit: partitions before `rem` each contributed one extra row to the running offset, so partition `id`'s start is shifted by `min(id, rem)`. Note that with `threads_ == 1` this returns $[0, \text{rows\_})$ — the full grid — so the sequential sweep is the degenerate single-partition case. The `RemainderRowsHandled` test (Section 11) targets exactly this logic.

7.5. Neighbour counting and the stencil

Código 18: src/engines/cpu/CpuEngine.cpp — both edge modes (condensed).

```
int CpuEngine::countNeighbors(const unsigned char* g, std::size_t x,
                              std::size_t y) const {
  int n = 0;
  if (wrap_) {                              // toroidal: indices wrap
    const long cols = (long)cols_, rows = (long)rows_;
    for (long dy = -1; dy <= 1; ++dy)
      for (long dx = -1; dx <= 1; ++dx) {
        if (dx == 0 && dy == 0) continue;       // a cell is not its own neighbour
        const std::size_t nx = ((long)x + dx + cols) % cols;   // +size before %
        const std::size_t ny = ((long)y + dy + rows) % rows;
        n += g[ny * cols_ + nx];
      }
  } else {                                  // bounded: OOB counts as dead
```

```
    for (long dy = -1; dy <= 1; ++dy)
      for (long dx = -1; dx <= 1; ++dx) {
        if (dx == 0 && dy == 0) continue;
        const long nx = (long)x + dx, ny = (long)y + dy;
        if (nx < 0 || ny < 0 || nx >= (long)cols_ || ny >= (long)rows_) continue;
        n += g[(std::size_t)ny * cols_ + (std::size_t)nx];
      }
  }
  return n;
}

void CpuEngine::computeRows(const unsigned char* src, unsigned char* dst,
                            std::size_t yBegin, std::size_t yEnd) const {
  for (std::size_t y = yBegin; y < yEnd; ++y) {
    const std::size_t row = y * cols_;          // hoisted out of inner loop
    for (std::size_t x = 0; x < cols_; ++x)
      dst[row + x] = nextState(src[row + x], countNeighbors(src, x, y));
  }
}
```

Cells are 0/1, so *summing* neighbour values *is* counting the live ones — no comparison needed. In the toroidal branch the `+ size` before `%` is necessary because C++'s `%` on a negative operand yields a negative result; adding the size first guarantees a non-negative dividend, so -1 mod cols becomes cols $-$ 1 (wrap to the far edge). In the bounded branch, out-of-range neighbours are simply skipped, i.e. counted as dead — this is the **default** edge mode. Both modes must match every backend bit-for-bit, which is why both are tested.

`computeRows` is the single implementation of the stencil, called by *both* the sequential path (full range) and every worker (its slice). One implementation of the math is the source of the sequential/parallel equivalence. Hoisting `row = y*cols_` out of the inner loop saves a multiply per cell.

7.6. upload, step, download

Código 19: src/engines/cpu/CpuEngine.cpp — the per-generation core.

```
void CpuEngine::upload(const Grid& initial) {
  rows_ = initial.rows();  cols_ = initial.cols();
  cur_.assign(initial.data(), initial.data() + initial.size());
  nxt_.assign(cur_.size(), 0u);
}

void CpuEngine::step() {
  Timer t;                                  // CPU's lastKernelMillis source
  if (workers_.empty()) {
    computeRows(cur_.data(), nxt_.data(), 0, rows_);  // SEQUENTIAL: one full sweep
```

```
  } else {
    src_ = cur_.data();  dst_ = nxt_.data();        // publish buffers to workers
    barrier_->arrive_and_wait();                   // phase 1: release workers
    auto [yBegin, yEnd] = rangeFor(0);
    computeRows(src_, dst_, yBegin, yEnd);          // main thread = partition 0
    barrier_->arrive_and_wait();                   // phase 2: wait for all
  }
  std::swap(cur_, nxt_);                          // the ping-pong
  lastMs_ = t.elapsedMillis();
}

void CpuEngine::download(Grid& out) {
  std::copy(cur_.begin(), cur_.end(), out.data());   // host copy; device->host on GPU
}
```

`upload` records the dimensions, copies the seed into `cur_`, and sizes `nxt_` (its initial contents do not matter — `step` overwrites every cell).

`step` is timed by the chrono `Timer`; this is the CPU's `lastKernelMillis()` source. In the **sequential** branch it is one `computeRows` over $[0, \text{rows\_})$ with zero synchronisation. In the **parallel** branch the main thread mirrors the worker handshake from its own side: publish the buffer pointers, phase-1 arrival releases the workers, the main thread computes partition 0 itself instead of idling, and phase-2 arrival blocks until every worker has finished. The symmetry is the point: `step` runs the same phase1→compute→phase2 dance that each worker runs, which is why they stay in lockstep generation after generation. The `std::swap` is the ping-pong: `nxt_` becomes the new `cur_`.

`download` copies the current buffer into the caller's grid. On the CPU this is the "no-op cost" the interface mentions — a host copy; on the GPU backends this same method will be the device→host transfer, which is precisely why it is excluded from the benchmark.

What would break.

Remove the `stop_` check and the destructor deadlocks. Swap the order of "publish `src_/dst_`" and "phase-1 arrive" and workers could read stale pointers. Give `nextState` any per-cell state and the sequential/parallel equivalence collapses — it is stateless on purpose.

8. The pattern pipeline — RleLoader.cpp and Pattern.cpp

A `Pattern` is a *sparse* seed: a bounding box plus the list of live `(x,y)` offsets. Sparse because that is the natural output of RLE decoding and it decouples a pattern from any particular grid size.

Código 20: src/patterns/RleLoader.cpp — the decode loop (core).

```
std::size_t x = 0, y = 0, count = 0;
```

```
bool haveCount = false;
for (const char ch : body) {                    // body = payload, newlines dropped
  if (std::isspace((unsigned char)ch)) continue;
  if (std::isdigit((unsigned char)ch)) {
    count = count * 10 + (std::size_t)(ch - '0');  // build multi-digit run lengths
    haveCount = true; continue;
  }
  const std::size_t n = haveCount ? count : 1;     // count defaults to 1
  bool done = false;
  switch (ch) {
    case 'b': x += n; break;                          // dead run: advance x
    case 'o': for (std::size_t i=0;i<n;++i) pat.liveCells.emplace_back(x++, y); break;
    case '$': y += n; x = 0; break;                    // end of row(s)
    case '!': done = true; break;                     // end of pattern
    default: break;                                  // ignore unexpected
  }
  count = 0; haveCount = false;
  if (done) break;
}
```

The RLE grammar is `<count><tag>` with the count defaulting to 1. Digits accumulate into `count`; on a tag, `n` is the accumulated count or 1, and the four tags drive a cursor: `b` skips dead cells, `o` emits `n` live cells (advancing `x`), `$` ends rows (and `n$` skips `n` blank rows at once — how RLE compresses empty space), `!` terminates. The payload is read into one `body` string with newlines removed first, because RLE tokens can wrap across lines and cannot be decoded line-by-line.

A concrete trace.

The `birth_on_six.rle` payload `3o$obo$o!` decodes as:

Código 21: Decoding 3ooboo! – six live cells around a DEAD centre (1,1).

```
3o   -> (0,0) (1,0) (2,0)     x=3 y=0
$    -> y=1 x=0
o    -> (0,1)              x=1
b    -> x=2                 (skips (1,1): the centre stays DEAD)
o    -> (2,1)              x=3
$    -> y=2 x=0
o    -> (0,2)              x=1
!    -> stop
```

That leaves a dead centre $(1, 1)$ surrounded by exactly six live neighbours — which under the variant rule must be *born*. This fixture is the payload behind the "birth on 6" correctness test (Section 11).

Two robustness details round out the loader: the header parser (`x = 3, y = 3, rule = B36/S23`) *ignores* the `rule=` field — the simulation rule lives in code, not data, so a stray

fixture cannot silently change the simulation — and if the header omits dimensions, the loader infers the bounding box from $\max(x)+1$, $\max(y)+1$ over the decoded cells. Opening a missing file throws `std::runtime_error`.

Código 22: src/patterns/Pattern.cpp — stamping, with clipping.

```
void Pattern::applyTo(Grid& grid, std::size_t originX, std::size_t originY) const {
  for (const auto& [dx, dy] : liveCells) {
    const std::size_t x = originX + dx, y = originY + dy;
    if (x < grid.cols() && y < grid.rows())   // clip, don't crash
      grid.at(x, y) = 1u;
  }
}
```

Because `x`, `y` are unsigned, the single test `x < grid.cols()` rejects both "too far right" and any wrap-around at once — no separate `>= 0` check is needed. Cells that fall outside are silently dropped, which is the right behaviour for a centering seeder: a pattern slightly larger than the grid still produces a valid, cropped board rather than a crash.

9. Rendering — NullRenderer, TextRenderer, AnsiRenderer

There are three renderers behind `IRenderer`, none of which is in the benchmark path (the loop only renders when the renderer is not Null). `NullRenderer::render` is an empty inline body — header-only because there is nothing to define. Its entire purpose is to *not* exist in the timed path (Section 6).

Código 23: src/render/TextRenderer.cpp — one buffer, one write.

```
void TextRenderer::render(const Grid& grid, std::uint64_t generation) {
  const std::size_t glyph = std::max(alive_.size(), dead_.size()); // bytes per cell
  std::string out;
  out.reserve(grid.size() * glyph + grid.rows() + 32);
  out += "generation "; out += std::to_string(generation); out += '\n';
  for (std::size_t y = 0; y < grid.rows(); ++y) {
    for (std::size_t x = 0; x < grid.cols(); ++x)
      out += (grid.at(x, y) ? alive_ : dead_);   // alive_/dead_ are std::string
    out += '\n';
  }
  out += '\n';
  std::fputs(out.c_str(), stdout);              // single write, no per-cell putchar
}
```

The mechanism worth noting is that it builds the entire frame into one `std::string` (pre-reserved so it never reallocates) and emits it with a single `fputs`. Per-cell `putchar` would

lock and flush the stream thousands of times per frame. `at(x, y)` keeps the `(column, row)` argument order matching `Grid`'s convention even though storage is row-major. The live-cell glyph defaults to the Unicode full block (U+2588) and the glyph fields are `std::string` rather than `char` so the multi-byte UTF-8 block fits; it is one display column wide, so the grid stays aligned with the one-column . dead cell. This renderer is for eyeballing correctness only — never in a benchmark.

9.1. AnsiRenderer — in-place animation, clipped to the viewport

`TextRenderer` *appends* each frame, so the terminal scrolls and any row wider than the terminal soft-wraps; it is a dump, not an animation. `AnsiRenderer` instead homes the cursor and overwrites the same region every frame using ANSI/DEC control sequences, producing a stable in-place animation. It is a small-grid visualisation aid; benchmarks still use `NullRenderer`.

Código 24: src/render/AnsiRenderer.cpp — the per-frame sequence (condensed).

```
// Construction: hide cursor, disable autowrap (wide rows clip, not wrap), clear.
//   "\033[?25l"  "\033[?7l"  "\033[2J"
// Destruction (RAII): restore "\033[?7h" (wrap on) "\033[?25h" (cursor on).
void AnsiRenderer::render(const Grid& grid, std::uint64_t generation) {
  unsigned termRows = 24, termCols = 80;
  terminalSize(termRows, termCols);                    // ioctl(TIOCGWINSZ), 80x24 fallback
  const std::size_t visRows = std::min(grid.rows(), termRows > 1 ? termRows - 1u : 1u);
  const std::size_t visCols = std::min(grid.cols(), termCols);   // clip to the viewport
  std::string out;
  out += "\033[?2026h";                                 // begin synchronized (atomic) update
  out += "\033[H";                                     // cursor home (overwrite, no flicker)
  out += "generation " + std::to_string(generation);
  if (visRows < grid.rows() || visCols < grid.cols())    // note the clip window
    out += " [" + std::to_string(visCols) + "x" + std::to_string(visRows) +
           " of " + std::to_string(grid.cols()) + "x" + std::to_string(grid.rows()) + "]";
  out += "\033[K\n";
  for (std::size_t y = 0; y < visRows; ++y) {
    for (std::size_t x = 0; x < visCols; ++x) out += (grid.at(x, y) ? alive_ : dead_);
    out += "\033[K\n";                                  // clear to EOL (wipe leftovers)
  }
  out += "\033[J";                                     // clear below (handles a shrunk terminal)
  out += "\033[?2026l";
  std::fputs(out.c_str(), stdout); std::fflush(stdout);
  if (delayMs_ > 0) std::this_thread::sleep_for(std::chrono::milliseconds(delayMs_));
}
```

Big grids: it clips, it does not wrap.

This is the key behavioural difference. `TextRenderer` relies on the terminal: a 300×300 board in an 80-column terminal wraps every row and scrolls — unreadable but harmless. `AnsiRenderer` cannot allow wrapping, because a wrapped row would occupy several terminal rows and desynchronise the cursor-home arithmetic that the whole in-place scheme depends on. So it disables autowrap (`\033[?7l`) and renders only the top-left $\min(\text{rows}, \text{term} - 1) \times \min(\text{cols}, \text{term})$ window, annotating the header, e.g. `generation 0 [80x23 of 300x300]`. The size is re-queried every frame, so resizing the terminal is handled.

Robustness details.

A per-frame `\033[?2026h/l` pair asks the terminal for a *synchronized* (tear-free) update and is silently ignored where unsupported; `\033[K` wipes any leftover from a previously wider frame and `\033[J` wipes rows orphaned by a shrunk terminal; the cursor is hidden during animation and restored (with autowrap) by the destructor even on an early break or exception. A small `delayMs` (default 50 ms) paces the otherwise-uncapped loop so the animation is watchable; it lives only in this renderer, never in the timed path. The size query is POSIX (`ioctl(TIOCGWINSZ)`); off a TTY it falls back to 80×24.

10. Build system — CMakeLists.txt

The build must produce a working CPU + text binary on a machine with no CUDA toolkit and no OpenCL. That requirement shapes the GPU targets.

Código 25: CMakeLists.txt — the core/CPU targets and the GPU double-gate.

```
add_library(gol_core STATIC
   src/core/Config.cpp src/patterns/Pattern.cpp src/patterns/RleLoader.cpp)
target_include_directories(gol_core PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/
    ↪ include)

find_package(Threads REQUIRED)                  # std::thread / std::barrier
add_library(gol_cpu src/engines/cpu/CpuEngine.cpp)
target_link_libraries(gol_cpu PUBLIC gol_core Threads::Threads)

add_executable(gol src/core/main.cpp)
target_link_libraries(gol PRIVATE gol_core gol_cpu gol_render)

option(BUILD_CUDA "Build the CUDA engine" OFF)
if(BUILD_CUDA)
   set(_cuda_src ${CMAKE_CURRENT_SOURCE_DIR}/src/engines/cuda/CudaEngine.
    ↪ cu)
   if(EXISTS ${_cuda_src})                      # second gate: source must exist
      enable_language(CUDA)
      add_library(gol_cuda ${_cuda_src} .../kernel.cu)
      target_link_libraries(gol_cuda PUBLIC gol_core)
```

```
    else()
        message(WARNING "BUILD_CUDA=ON but source does not exist yet; skipping.")
    endif()
endif()
```

Each GPU backend is **double-gated**: an `option(... OFF)` *and* an `if(EXISTS <source>)` check. Even passing `-DBUILD_CUDA=ON` today emits a warning and skips the target because the `.cu` source does not exist yet — it does not fail configuration. The OpenCL target follows the identical pattern. `find_package(Threads REQUIRED)` is what makes `std::thread`/`std::barrier` link on the CPU engine.

Código 26: Building and testing.

```
cmake -S . -B build              # core + CPU + tests (GPU OFF by default)
cmake --build build
ctest --test-dir build --output-on-failure
# GPU opt-in (once the sources exist):
cmake -S . -B build -DBUILD_CUDA=ON -DBUILD_OPENCL=ON
```

The test `CMakeLists` prefers a system GoogleTest via `find_package(GTest CONFIG)` and otherwise `FetchContents` v1.17.0; it defines `PATTERNS_DIR` so tests find the RLE fixtures regardless of working directory, and registers each test with `gtest_discover_tests`.

11. Tests — the executable specification of the invariants

The tests are not an afterthought; they encode the invariants from Section 2 as runnable checks, in dependency order (`CpuEngine` is the oracle, so it is verified first).

- **`compile_smoke_test.cpp`** includes *every* interface header (so the scaffolding stays build-clean and mutually consistent) but deliberately does not call the declared-but-undefined symbols, so it links without those translation units. Its `VariantRule` case pins the rule truth table directly.
- **`rules_test.cpp`** checks `CpuEngine` on known patterns: a block is a still-life, a blinker oscillates with period 2, and — the defining case — a dead cell with exactly six live neighbours is born.
- **`cpu_parallel_test.cpp`** proves the central design claim. On a deliberately non-square, non-thread-divisible 128×130 grid it asserts thread counts $\{2, 3, 4, 8, 0\}$ all match the single-thread reference *bit-for-bit*, in both edge modes; a second case targets `rangeFor`'s remainder logic with 17×9 and counts $4, 5$.
- **`rle_loader_test.cpp`** covers parsing, dimensions, specific coordinates (including that `birth_on_six`'s centre is dead), the missing-file throw, and stamping with clipping.

Código 27: tests/cpu_parallel_test.cpp — the equivalence assertion.

```
const Grid reference = runN(start, 1, wrap, 25);        // sequential oracle
for (unsigned threads : {2u, 3u, 4u, 8u, 0u /* all hw cores */}) {
  Grid parallel = runN(start, threads, wrap, 25);
  EXPECT_TRUE(parallel == reference)
      << "threads=" << threads << " wrap=" << wrap << " diverged";
}
```

The fourth test is still pending.

Cross-backend equivalence — seed CPU/CUDA/OpenCL identically, run N generations, assert bit-for-bit equality against the `CpuEngine` oracle — waits on the GPU engines. An external oracle is also available: the headless `bgolly` runner simulates `B36/S23` on an infinite grid, and the `highlife_replicator.rle` / `highlife_spaceship.rle` fixtures already reproduce it bit-for-bit, so they make good large-grid cross-checks for the GPU backends.

12. Timing methodology and the expected GPU story

12.1. The metric

The headline number is fixed across all three backends:

$$\text{Mcells/s} = \frac{\text{rows} \times \text{cols} \times \text{generations}}{t_{\text{elapsed}} \times 10^6} \tag{1}$$

measured against `NullRenderer` so output and transfer cost never enter t. Timing is uniform and in-program (not from a profiler) so the three backends are measured the same way, exposed through `ISimEngine::lastKernelMillis()`: `std::chrono` on CPU, `cudaEvent_t` around the launch on CUDA, and command-queue events with `CL_QUEUE_PROFILING_ENABLE` on OpenCL. Kernel time is reported separately from host $\leftrightarrow$ device transfers, and the speed-up is

$$S(N) = \frac{T_{\text{CPU}}(N)}{T_{\text{GPU}}(N)}. \tag{2}$$

12.2. What the numbers should show, and why

This is a memory-bound, arithmetically trivial nine-point stencil over one-byte cells. That single fact predicts the shape of every curve the report will produce:

Block size gives a concave curve that plateaus by 128–256.

Below that, blocks are too small to hide memory latency (too few warps in flight to cover the stalls of a bandwidth-bound kernel); past it, more threads per block do not buy more

effective bandwidth, so the curve flattens. The mechanism is occupancy versus available memory throughput, not arithmetic.

Shared/local memory may give little or even negative gain.

The classic stencil optimisation stages a tile plus a halo into shared memory to reuse neighbour reads. Here each cell is one byte, so the working set of a tile already lives in L1/L2; the explicit staging adds index arithmetic and a barrier without saving meaningful DRAM traffic, and can lose. Explaining *why* this optimisation underperforms is one of the more interesting parts of the report.

No optimisation helps at small grids.

Kernel-launch and transfer overhead dominate when there are few cells, so $S(N) < 1$ for small N and the CPU is competitive or faster. The sweeps must therefore run at large $N \times M$ to see the GPU win. Identifying the threshold N where $S(N) > 1$ sustainably is a graded deliverable.

The deep-dive profiling targets the *best* parallel solution only: `ncu` (Nsight Compute) for CUDA kernel-internal metrics — occupancy, stalls, memory throughput (it will not profile OpenCL) — and `nsys` (Nsight Systems) for the system timeline, which can also trace OpenCL on NVIDIA hardware.

13. Status, roadmap, and how to extend

Done.

The backend-agnostic core, the CPU engine (sequential and parallel in one path), both renderers, the RLE pipeline, the CLI, and the first three test suites. The CPU engine is the reference oracle.

Pending.

`CudaEngine` (`src/engines/cuda/`) and `OpenCLEngine` (`src/engines/opencl/`); the cross-backend equivalence test; and the benchmark sweeps, results CSVs, and analysis notebooks that populate `main.tex`.

How a GPU backend slots in.

Implement `ISimEngine`: own two device buffers, `upload` once, ping-pong on the device inside `step`, `download` only when rendering. Reuse `nextState` from `LifeRules.hpp` (CUDA includes it directly; OpenCL mirrors it in `kernel.cl`). Match the edge handling of `countNeighbors` exactly, in both bounded and toroidal modes. Wire it into `main.cpp`'s engine selection and add the cross-backend test against the CPU oracle. Because the `Grid` layout and the metric are already fixed, a correct new backend needs no changes to the core, the harness, or the report's measurement code.

14. Keeping this manual in sync

This manual, the graded report (`main.tex`), and `CLAUDE.md` are treated as living documents. The rule, recorded in `CLAUDE.md` under "Documentation maintenance", is: whenever a source file is added, removed, or changed in a way that alters behaviour or structure, update all three in the same change — this manual's relevant section and file map (Table 1), the report if results or methodology move, and `CLAUDE.md`'s "Current state" and layout. The most common trigger will be landing the CUDA or OpenCL backend: that touches Sections 4.2, 7 (as the oracle reference), 11, 12, and 13 here, plus the implementation and results sections of the report.